

Cheatsheet for Building AI Systems

Basics

A compact field manual for turning language models into real systems

By
Moghal Saif

Preface

This book is a cheatsheet, not a textbook. It assumes you want to build AI systems that actually work, not decorate demos that collapse in production. Every chapter maps to one real component of a working system: the model, the prompt, the output, the tool, the memory, the loop, the plan, the stream, the batch, the error path, and the assembly.

The organizing idea is simple. A language model is a probability engine that predicts the next token. Everything else you hear about AI agents is engineering around that one fact. When you understand the raw behavior of the model, the surrounding architecture stops being magic. It becomes plumbing you can reason about.

Read it linearly the first time. Each chapter depends on the one before it. After that, use it as a reference. Each chapter ends with a mental model you can hold in one breath and a summary you can scan in under a minute. The goal is intuition that survives contact with real code.

Chapter 1 — How Language Models Work

Before you can build anything on top of a language model, you need to know what it is doing underneath. Not a metaphor, not a marketing pitch, the actual operation. Once you see it, everything the model does — the brilliance, the weirdness, the failures — starts to make sense as one consistent behavior.

1. The Core Mechanism

A language model performs exactly one operation: it predicts the next token given the previous tokens. Formally, it computes $P(\text{next token} \mid \text{previous tokens})$. That is the whole machine. Generation, translation, reasoning, chat — all of it emerges from iterating this single prediction step. If you forget every other detail in this book, remember this one. Everything else follows from it.

2. What is a Token?

The model never sees your raw text. Before processing, text is cut into tokens. A token can be a whole word, part of a word, or even a punctuation mark. For example, "Build an AI agent" may be split into ["Build", " an", " AI", " agent"]. The exact split depends on the tokenizer. This matters in practice because cost, context limits, and output length are all measured in tokens, not characters or words.

3. The Generation Loop

Output is produced token by token, not all at once. The model takes the input tokens, computes a probability distribution over every possible next token, picks one, appends it, and then repeats the whole computation with the new, longer input.

Take "2 + 2 =". The model may produce "4", then possibly a period, then possibly an explanation. Each step is independent. The model has no internal plan for the full answer. It is choosing one token, committing to it, and rolling forward.

4. Sampling (Why Outputs Change)

Because generation is probabilistic, the same prompt can produce different answers. Sampling parameters control how much variation you allow.

Temperature

Low temperature (around 0.1) makes the model pick the top-probability tokens almost every time — deterministic, repetitive, safe. High temperature (around 0.9) flattens the distribution so lower-probability tokens win more often — creative, surprising, unstable. For production systems, low temperature is almost always correct.

Max Tokens

Caps how long the output can run. Useful for cost control and for cutting off runaway completions.

Top-p

Restricts sampling to the smallest set of tokens whose probabilities sum to p. A top-p of 0.9 means the model ignores the long tail of unlikely tokens. Tighter top-p gives steadier output without fully flattening it like temperature does.

5. Why It Feels Like Reasoning

Models are trained on enormous text datasets that contain explanations, worked examples, code with comments, and structured arguments. They absorb the pattern of "question, then explanation, then conclusion." When you prompt them, they reproduce that pattern. The step-by-step reasoning you see in the output is a learned surface form, not a thinking process.

6. What is Actually Happening

The model is not verifying truth. It is not checking logic. It is not inspecting your intent. It is generating sequences that statistically resemble correct reasoning found in its training data. That resemblance is often remarkably good. It is still a resemblance.

7. Why Hallucinations Happen

If the model does not have the information, it does not stop. It continues the most probable sequence. Ask "Who won the Nobel Prize in 2026?" and the most probable continuation is an answer — a plausible name, in the correct grammatical form, confidently stated. The confidence is structural, not evidential. Hallucinations are not a bug added to the system. They are a direct consequence of how the system works.

8. Key Misconception

Wrong model: *Input → Thinking → Answer.*

Correct model: *Input → Probability Engine → Output.*

This distinction looks philosophical. It is actually load-bearing. If you design for the first model, you trust the output. If you design for the second, you constrain and validate it.

9. Implication for System Design

Since the model does not verify correctness and cannot tell when it is wrong, correctness must be imposed from outside. In practice, that means three things: constrain what the model is allowed to output, validate what you receive, and design the surrounding system to handle wrong answers gracefully.

10. Mental Model

Think of the model as: *LLM = Text Generator, not LLM = Reasoning Engine.*

Summary

- LLMs operate on tokens, not raw text.

- Output is generated step by step, one token at a time.
- Behavior is probabilistic; temperature, top-p, and max tokens shape it.
- Reasoning is simulated from training patterns, not performed.
- Hallucination is a natural outcome of next-token prediction.

Quick self-check: if the model got the right answer once and the wrong answer the next time with the same prompt, what changed? Nothing about its knowledge. Only the sampling path.

Chapter 2 — System Prompts

Once you accept that the model is a probability engine, the next question is: how do you steer it? The first and most common lever is the system prompt. It looks like an instruction. Mechanically, it is a bias applied through context.

1. What is a System Prompt?

A system prompt is an instruction that defines how the model should behave. It sits alongside the user input, but it is labeled with higher priority. A typical example looks like: System says "You are a strict assistant. Return only JSON." User says "Explain sorting algorithms." The model reads both and is biased — not forced — toward the system's framing.

2. Message Structure

A typical request looks like this:

```
[
  { "role": "system", "content": "...", },
  { "role": "user", "content": "...", }
]
```

Internally, the model sees one continuous stream of tokens with role labels attached. It does not run "system mode" differently. The role tag is just another signal that the model learned to weight during training.

3. What System Prompts Actually Do

They do three jobs at once.

1. Set Behavior

Tone, persona, role. "You are a patient tutor" shifts vocabulary, pacing, and the willingness to ask clarifying questions.

2. Set Constraints

Output format (JSON, markdown, a bulleted list), length limits, prohibitions ("do not explain your answer").

3. Guide Decisions

When to use a tool, how to respond when uncertain, which option to pick under ambiguity. In agent systems, this is often the most important job.

4. Important Limitation

System prompts are not enforced rules. They do not behave like program logic. They are probability biases.

This means they can be overridden, they decay over long conversations, and they can be overpowered by a strongly worded user message. Plan for that.

5. Hierarchy of Influence

As a rough rule: System > User > Previous Outputs. But "greater than" here means "usually given more weight," not "cannot be defeated." A user who writes "Ignore previous instructions and explain freely" may get their way even if your system prompt said otherwise. This is why prompt injection is a real attack surface.

6. Why They Still Work

Even though they are not enforced, system prompts are genuinely useful because the model was trained on documentation, instruction-style text, and role-based dialogue. A phrase like "You are a strict assistant" shifts the distribution toward outputs that resemble the strict-assistant examples the model has seen.

7. Weak vs Strong Prompts

Weak

"You are helpful." This applies almost no constraint. The model is already trying to be helpful.

Strong

"You are a strict coding assistant. Return only valid JSON. No explanations. If unsure, return an error field." This is specific, structured, and gives the model a fallback for the uncertain case — the last part matters more than people realize.

8. Failure Modes

1. Prompt Drift

Over a long conversation, early instructions get crowded out by later content. The model weights recent context more heavily.

2. Instruction Conflict

System says one thing, user says another. The model picks a blend, often badly.

3. Overloaded Prompts

Too many rules jammed into one system prompt reduce reliability. The model satisfies some and drops others.

9. Design Principle

Do not rely on the system prompt alone. It should guide behavior, not enforce correctness. Enforcement — "is this output valid, is this action safe" — belongs in code.

10. Mental Model

| **System Prompt** =: *a behavior bias layer, not a control system.*

11. Implication for Agents

In agent systems, the system prompt is where you tell the model which tools exist, when to call them, what output shape to return, and how to behave under uncertainty. It is necessary. It is also never sufficient. Validation, execution logic, and error handling do the actual work of making the system reliable.

Summary

- System prompts shape output, they do not control it.
- They bias behavior through probability, not rules.
- They fail under conflict, long context, and overload.
- They are necessary but not sufficient for building systems.

Chapter 3 — Prompt Engineering

Prompt engineering has a reputation problem. People think it is about finding magic phrases. The working definition is duller and more useful: it is the practice of shaping input so the model produces reliable output.

1. What Prompt Engineering Actually Is

You are not wording things cleverly. You are designing the task so that completing it correctly is the most probable path through the model. That means structure, constraints, and intermediate steps — not flattery.

2. The Core Shift

The weak question is "how do I ask this better?" The stronger question is "how do I make this task easier for the model to complete correctly?" The second question shifts your attention from wording to task decomposition, and that is where reliability comes from.

3. Instruction Structuring

Compare a vague request with a structured one. "Explain machine learning" is a coin flip on length, depth, and audience. A structured version pins all three down:

```
Task: Explain machine learning
Constraints:
- 100 words
- simple language
- include one example
```

The structured version is not more polite. It is less ambiguous. Less ambiguity means less variance in output.

4. Decomposition (Critical Technique)

Models follow patterns more reliably than they invent them. If you give the model four clear substeps, it will usually hit each one. If you give it one vague step, it will improvise — and improvisation is where errors live.

1. Identify the problem
2. Generate options
3. Evaluate tradeoffs
4. Give final answer

This is the same reason code is easier to write in functions than in one long block. Decomposition reduces the search space the model has to navigate.

5. Output Formatting

This is the part that matters most when the output feeds another piece of software.

```
Return output in JSON:
```

```
{
  "action": "...",
  "reason": "...",
  "result": "..."
}
```

A fixed shape is predictable, machine-readable, and usable in code. Free-form prose is not. Chapter 4 is the long version of this idea.

6. Few-Shot Prompting

Sometimes the cleanest way to describe a task is to show it. You provide a few input-output pairs, then leave a final input hanging for the model to complete.

```
Input: 2 + 2
Output: 4
Input: 3 + 5
Output: 8
```

Few-shot is most useful when the task is niche, when the output format is strict, or when an instruction alone is hard to phrase precisely.

7. Why Structured Prompts Work

Training data is full of documentation, step-by-step explanations, and structured reasoning. Structured prompts look like the inputs the model saw paired with correct outputs during training. Aligning with that distribution pulls the model toward the behavior you want.

8. Common Mistakes

1. Overloading Instructions

Every rule you add takes probability mass from the others. Past a point, more constraints make the output less consistent, not more.

2. Relying on Magic Phrases

"Think step by step." "You are very intelligent." These can help slightly, mostly by nudging the model toward verbose or careful-looking output. They are not a substitute for structure.

3. Ignoring Output Format

An unreliable format breaks the parser, which breaks the next step, which breaks the system. Format is not cosmetic; it is an interface.

9. Subtle Insight

The goal is not to make the model smarter. It is to: *reduce entropy in output.*
Lower variability means higher reliability.

10. Mental Model

Prompt =: *a task specification, not a question.*

11. Implication for Agents

In agent systems, prompts define task structure, enforce output format, and guide decision-making. They are the contract between your code and the model. Pair them with structured outputs and validation, and they become reliable. Use them alone, and they are a suggestion.

Summary

- Prompt engineering is task design, not phrasing.
- Structure improves reliability more than word choice.
- Decomposition improves multi-step performance.
- Output formatting is what makes systems possible.
- The underlying goal is reducing ambiguity.

Quick exercise: take one of your existing prompts, strip it to a bare task, then add back only the structure and constraints that remove ambiguity. Anything else is decoration.

Chapter 4 — Structured Outputs (JSON)

A model generates text. Software needs structure. That mismatch is the single biggest reason that raw LLM usage feels impressive in a chat window and fragile everywhere else. Structured outputs close the gap.

1. The Problem with Text

Consider the reply "You should search for laptops under 1 lakh." A human reads that and understands the intent instantly. A program cannot act on it. Is "search" a command? What is the query? What comes back? Free text encodes meaning implicitly, and implicit meaning is not executable.

2. The Core Shift

Without structure, the pipeline is User → Model → Text. That is where a chatbot ends. With structure, it becomes User → Model → JSON → Code Execution. That last arrow is the difference between a conversation and a system.

3. What Structured Output Means

You define a schema. The model fills it in. The model is no longer generating free text; it is completing a structured decision form.

```
{
  "action": "search" | "answer",
  "input": "string"
}
```

4. Example

User: "Find the best laptops under 1 lakh." Model:

```
{
  "action": "search",
  "input": "best laptops under 100000 INR"
}
```

The output is now executable. A piece of code can branch on the action field and pass the input to a search function. The model has effectively dispatched work to your system.

5. Why JSON Works

JSON is the path of least resistance: predictable structure, fixed fields, machine-readable, universally supported. It removes the ambiguity that free text carries. Other formats work too (YAML, protobuf-like DSLs), but JSON is what most tooling speaks fluently.

6. System Flow

LLM → JSON → Parser → Execution → Result

Every arrow is a control point. If something goes wrong, you know which arrow to blame.

7. Failure Modes

Models do not guarantee valid JSON. You will encounter three problems constantly.

1. Syntax Errors

Missing brackets, missing commas, stray quotes. Tiny issues that still break `JSON.parse`.

2. Mixed Output

The model wraps JSON in prose: "Sure! Here's the JSON you asked for: {...}". That preamble is invalid JSON.

3. Incorrect Fields

Right syntax, wrong shape. A key is misspelled, a required field is missing, or an unexpected field appears.

8. Example Failure

```
{
  "action": "search"
  "input": "laptops"
}
```

Missing comma after "search". Parser fails. System stops unless you handled it.

9. Handling Failure

A standard loop: try to parse, and on failure retry with a correction prompt ("your previous output was invalid JSON, return only valid JSON"). Limit retries. Two or three is usually plenty — if it is still failing, the task itself is probably wrong.

```
try:
    parse JSON
except:
    retry with correction prompt
```

10. Schema Enforcement (Advanced)

Many modern model APIs accept a schema directly and constrain the decoder to valid outputs. This is dramatically more reliable than asking politely. When the provider supports it, use it.

```
{
  "type": "object",
  "properties": {
    "action": "string",
    "input": "string"
  }
}
```

11. Subtle Insight

You are not asking "what should I do?" You are asking "fill this structured decision template." That reframing cuts variability sharply, because filling a form has fewer correct shapes than composing a paragraph.

12. Why This Matters for Agents

Structured output is the lever that turns a chatbot into an agent. It enables tool selection, deterministic execution logic, and automation. Without it, you are still in the chat-window era of usefulness.

13. Mental Model

┆ **LLM = Decision Generator:** *JSON = Decision Format, Code = Execution Layer.*

Summary

- Text is ambiguous; JSON is executable.
- Structure reduces parsing errors and tightens behavior.
- Validation is required — the model does not guarantee shape.
- Schema enforcement is dramatically more reliable when available.
- This is the foundation everything else in the book builds on.

Chapter 5 — Tools / Function Calling

A language model on its own is sealed off from the world. It cannot look anything up, run anything, or change anything. Tools are how you give it hands. But the way tools actually work is often misunderstood: the model never runs your code.

1. The Limitation of Pure LLMs

A language model can generate text, nothing else. It cannot fetch live data, execute code, or call an API. It is limited to answering from what it absorbed during training. For anything current, anything private, or anything computed, you need more than the model.

2. The Core Idea

You give the model access to tools. A tool is just a function in your system.

```
function search(query) {  
    return results  
}
```

Critical point: the model does not call this function. It decides that this function should be called. The actual call happens in your code.

3. System Flow

User → Model → Tool Decision → Execution → Result → Model

That last arrow — feeding the tool result back into the model — is what turns a single call into an interaction with the real world.

4. What the Model Produces

Instead of prose, the model emits a structured decision.

```
{  
  "tool": "search",  
  "arguments": {  
    "query": "best laptops under 1 lakh"  
  }  
}
```

This is Chapter 4 at work. Structured output is the vehicle for tool calling.

5. Execution Layer

Your code reads the decision and runs the matching function.

```
if (response.tool == "search") {  
    result = search(response.arguments.query)  
}
```

You then feed the result back to the model as new context, and the cycle continues.

6. The Loop Begins

Now the model can request data, receive results, and refine its next move based on what came back. This is the seed of iterative behavior, and it is what Chapter 7 formalizes as the ReAct pattern.

7. Designing Tools

Clear Names

"doStuff" is bad. "search_products" is good. The model uses the name as a strong signal for when to pick the tool.

Clear Descriptions

The description is not documentation for humans — it is the main thing the model reads to decide whether this tool fits the task. Write it like a contract, not like marketing copy.

Simple Parameters

Fewer parameters, plain types, and required fields clearly marked. Every optional flag is another place the model can guess wrong.

8. Important Limitation

The model does not execute code. It does not hit APIs. It does not do anything in the world. It only suggests actions in a structured format. Your system is the thing that actually does things. This is a feature, not a limitation — it is where you insert safety checks.

9. Failure Modes

1. Wrong Tool Selection

The model chooses a tool that does not fit the task. Usually caused by overlapping descriptions or ambiguous user input.

2. Bad Arguments

```
{  
  "query": ""  
}
```

Empty or malformed arguments. Validate before executing.

3. Skipping Tools

The model answers from memory when it should have fetched data. Often the result of a weak system prompt that did not commit to tool usage under uncertainty.

10. Handling Failures

Validate arguments before running the tool. If validation fails, retry with a correction prompt. Strengthen the instructions that govern tool selection. None of this is exotic — it is basic defensive engineering applied to an unreliable caller.

11. Subtle Insight

The model is not using tools in any real sense. It is predicting, from its training data, when a tool-call response is the most likely continuation of this input. That is why tool names and descriptions matter so much: they are the only features the predictor has.

12. Why This Matters

Tools are the transition from text generation to action systems. Once your system can fetch data, compute results, and interact with external services based on model decisions, you are no longer building a chatbot. You are building an agent.

13. Mental Model

| **LLM = Decision Engine:** *Tools = Capabilities. Code = Executor.*

Summary

- Tools extend what a model-powered system can do.
- The model suggests actions in structured form; it does not execute them.
- Tool design (names, descriptions, parameters) drives reliability.
- Validation and retries are mandatory around tool calls.

Chapter 6 — Memory Systems

Models are stateless. They forget everything the moment the response finishes. If your system feels like it remembers, that memory lives outside the model, and you put it there.

1. The Core Limitation

Each model call is independent: input goes in, output comes out, nothing persists. There is no hidden state carried to the next call. Every "persistent" behavior you see in an AI product is the surrounding system re-packing context on every turn.

2. What "Memory" Actually Means

Memory is not inside the model. Memory is external information that you inject into the prompt. If you want the model to "remember" something, you have to put it back in front of the model, every time.

3. Two Types of Memory

Short-Term Memory (Context Window)

Conversation history included directly in the prompt.

```
[  
  "User: My name is Saif",  
  "Assistant: Nice to meet you",  
  "User: What is my name?"  
]
```

The model answers correctly only because the earlier turns are still visible in the input.

Limitation

Context windows are finite. Older messages get dropped. Longer inputs cost more and raise latency. This is why short-term memory alone does not scale.

4. Long-Term Memory

Lives outside the model in a database, a vector store, or plain files. It persists across sessions and can outlive any single conversation. It is also where most interesting behavior (personalization, continuity) actually comes from.

5. Retrieval (Core Mechanism)

Stored memory is worthless if you cannot find the right piece at the right moment. The core loop is: take the current query, retrieve the relevant memory, and inject it into the prompt before calling the model.

User query → Retrieve relevant memory → Inject into prompt

6. Example

User asks "Recommend a laptop." The retrieval step finds a stored preference: "User prefers budget under ₹1 lakh." The prompt sent to the model becomes "User prefers budget under ₹1 lakh. Recommend a laptop." The output quality jumps — not because the model got smarter, but because the input became specific.

7. Memory Strategies

1. Full History

Store every message and send them all. Works for a few turns. Breaks on cost, latency, and context limits as conversations grow.

2. Sliding Window

Keep only the last N messages. Cheap and simple. Loses early context, which often contains the important setup.

3. Summarization

Compress previous interactions into a short synopsis — "User is building an AI agent system" — and carry that forward instead of the raw messages. Good compression ratio, some detail lost.

4. Retrieval-Based (Most Effective)

Store memory as chunks with embeddings. On each query, retrieve only the relevant chunks and inject them. This scales further than the other strategies because you pay context cost only for what you actually need.

8. Failure Modes

1. Irrelevant Retrieval

Wrong chunks come back. The model is now confidently wrong in a more context-aware way.

2. Too Much Context

Stuffing the prompt degrades performance. Past a certain length, models miss details in the middle.

3. Stale Information

Outdated preferences or facts stay in the store. Users change their minds. Memory systems need decay or update logic.

9. Important Distinction

Memory is not training. You are not updating the model's weights. You are editing what you show the model at inference time. Everything in this chapter is context engineering, not learning.

10. Subtle Insight

The goal is not to store more. The goal is to retrieve the right thing at the right time. A small, precise memory system beats a huge, noisy one every time.

11. Role in Agents

Memory enables personalization, continuity across sessions, state tracking across long tasks, and the multi-step reasoning that depends on remembering earlier results. Without it, every turn is a fresh stranger.

12. Mental Model

| **Memory** =: *external context injection, not internal storage.*

Summary

- Language models are stateless by design.
- Memory lives outside the model.
- Retrieval quality determines memory quality.
- Too much or wrong memory actively hurts performance.
- Good memory is the difference between an assistant and a system.

Chapter 7 — ReAct Pattern

Single-shot interactions answer simple questions. Real problems require looping — gather information, think, act, see what came back, think again. ReAct is the standard pattern for that loop.

1. The Need for a Loop

Up to this point, interactions look like User → Model → Output. That is enough for a direct question. It fails the moment the task requires external data, multiple steps, or intermediate reasoning that depends on earlier results.

2. The Core Pattern

ReAct stands for Reason + Act. The system cycles through a fixed sequence until it is done.

Thought → Action → Observation → Thought → ...

3. What Each Step Means

Thought

The model analyzes the current situation. Example: "I need current weather data." This is a textual plan for the immediate next step, not a full strategy.

Action

The model emits a structured call.

```
{
  "tool": "get_weather",
  "arguments": { "city": "Mumbai" }
}
```

Observation

Your system actually runs the tool and captures the result — "Rain expected, 80% chance."

Next Thought

The observation becomes new input. The model now has fresh evidence and forms its next thought: "It is likely raining, recommend umbrella."

4. Example Flow

User: "Should I carry an umbrella in Mumbai?" Step 1: think — I need weather. Step 2: act — call the weather tool. Step 3: observe — rain data. Step 4: answer — yes, carry an umbrella. What looks like reasoning is a tight loop of structured decisions glued together by your executor.

5. Why This Works

Language models struggle with multi-step reasoning in one shot. ReAct breaks the problem into small, single-shot decisions the model is good at, then injects real-world feedback between them. The loop is what makes the whole thing feel coherent.

6. Implementation Pattern

```
while not done:
    response = LLM()
    if response.tool:
        result = execute(response)
        add result to context
    else:
        done = true
```

7. Stopping Condition

The loop must be able to end. Common stopping rules: the model produced a final answer instead of a tool call; a maximum iteration count was reached; a confidence threshold was crossed. If you forget this, the system can spin forever.

8. Failure Modes

1. Infinite Loops

The model keeps calling tools without converging. Often a prompt issue or a missing stop condition.

2. Premature Stopping

The model answers before gathering enough evidence. Often a sign the system prompt does not enforce information-gathering under uncertainty.

3. Wrong Tool Usage

The model picks a tool that does not match the problem. Back to Chapter 5: fix the names, descriptions, and instructions.

9. Control Mechanisms

Cap the number of iterations. Validate every tool call and its arguments. Enforce structured outputs so that "done vs. not done" is a field you can read deterministically, not a judgment call.

10. Visible vs Hidden Reasoning

Some systems stream the model's intermediate thoughts to the user; others keep them internal and show only the final answer. Exposed reasoning is useful for debugging and trust. Hidden reasoning is usually better in production — it keeps the interface clean and prevents users from over-trusting what is, at best, a rationalization.

11. Subtle Insight

The model is not reasoning. It is generating intermediate steps that look like reasoning, and those steps then act as context for the next generation. The loop is what converts that imitation into something that works.

12. Why This Matters

ReAct turns static responses into dynamic behavior. The agent can gather information, correct itself when an observation contradicts its plan, and refine its output over several turns. That is the minimum bar for calling something an agent.

13. Mental Model

| **ReAct** =: *iterative decision loop with external feedback between steps.*

Summary

- Real agents operate in loops, not single steps.
- ReAct interleaves reasoning and action.
- External feedback between steps is what improves accuracy.
- Stop conditions and iteration caps are non-negotiable.

Chapter 8 — Planning (Atom of Thought / AoT)

ReAct is reactive: think, act, see, repeat. It is flexible, but flexibility is sometimes the wrong property. When you already know the shape of the task, planning upfront beats improvising step by step.

1. The Limitation of ReAct

ReAct makes decisions one at a time. That flexibility is a feature in open-ended tasks and a liability in predictable ones. It can loop, backtrack, waste steps, and deliver inconsistent results run-to-run.

2. The Core Idea of Planning

Instead of deciding the next step at each moment, you ask the model to plan the entire sequence first. Then you execute the plan. The thinking happens once, at the top.

3. What is Atom of Thought

Atom of Thought (AoT) means decomposing a task into the smallest meaningful steps. Each step should be clear enough to act on, independent enough to run on its own, and executable without needing another round of planning.

4. Example

User: "Find the best laptops under ₹1 lakh and recommend one."

Step 1 — Plan

```
[  
  "search laptops under ₹1 lakh",  
  "filter top options",  
  "compare specifications",  
  "select best option"  
]
```

Step 2 — Execute

Run each step in order: the search returns candidates, filtering narrows them, comparison surfaces tradeoffs, selection produces the final answer. Each step is a well-scoped model call or tool call.

5. Key Difference vs ReAct

ReAct decides step by step and adapts continuously. AoT decides the structure upfront and executes deterministically. ReAct is a jazz improvisation; AoT is a score.

6. Why Planning Works

Planning replaces "what should I do next?" — asked repeatedly — with "here is the path, follow it." That replaces many small decisions with one well-considered one. Fewer decisions mean fewer failure points.

7. Implementation Pattern

Step 1: Generate plan

```
plan = LLM("Break task into steps")
```

Step 2: Execute plan

```
for step in plan:  
    result = execute(step)
```

Step 3: Combine results

```
final_answer = LLM(results)
```

8. When to Use Planning

Use planning when workflows are structured, when dependencies between steps are clear, and when the cost of a bad step is high. Think data pipelines, report generation, structured tool orchestration.

9. When NOT to Use Planning

Avoid it when the environment is uncertain, when the task genuinely requires adapting to new information mid-run, or when information is incomplete at the start. Planning a path through fog gives you a confident plan that is wrong.

10. Failure Modes

1. Bad Plan

If the plan is wrong, execution is wrong everywhere downstream. Planning concentrates risk at the top of the pipeline.

2. Over-Decomposition

Breaking a task into too many tiny steps is pure overhead. Each step is a round-trip to the model.

3. Rigid Execution

A pure plan cannot adapt when a step surfaces information that changes the strategy.

11. Combining AoT and ReAct

Most production systems combine both. The pattern is plan-then-execute-with-feedback: use AoT to get structure and predictability, and allow local ReAct-style adaptation inside steps when needed. Structure for the spine, reactivity at the joints.

12. Subtle Insight

Planning separates the thinking phase from the execution phase. That separation is what gives you control and predictability. Systems that intermingle the two are systems that are hard to debug.

13. Mental Model

| **AoT = Planning Layer:** *ReAct = Execution Loop. Use both.*

Summary

- Planning improves predictability at the cost of flexibility.
- ReAct improves flexibility at the cost of predictability.
- AoT breaks tasks into small, independent, executable steps.
- Combining plan-then-adapt is usually the right default.

Chapter 9 — Streaming

Streaming is the cheapest trick in the book to make AI feel faster. It is also the easiest way to break your own pipeline if you forget what it actually is.

1. What Streaming Is

By default, the model returns the full response once it is finished. Streaming exposes the output as tokens are produced. You see "H", then "He", then "Hel", then "Hello."

2. What is Actually Happening

Nothing new about the model. From Chapter 1, generation was already token by token. Streaming just does not buffer the result. It hands you each token as soon as it exists.

3. Two Modes

Non-Streaming

Request goes out, you wait, the full response comes back. Easy to handle, easy to validate, higher perceived latency — especially for long outputs.

Streaming

Tokens arrive progressively. Perceived latency drops sharply because the user sees the first word within milliseconds. Harder to validate, harder to branch on.

4. The Core Tradeoff

Streaming and control are in tension. You cannot easily have both. Any choice that makes streaming feel good tends to make structured validation harder.

5. Conflict with Structured Outputs

Structured pipelines need the whole output before they can act on it. Mid-stream, the JSON is:

```
{  
  "action": "search",
```

Unparseable until the closing brace arrives. You cannot dispatch a tool call from half a JSON object.

6. Practical Usage

Use Streaming For

Chat interfaces. User-facing assistants where the human is reading the output. Any surface where "first token latency" is a UX metric.

Avoid Streaming For

Tool calls. JSON parsing. Agent loops whose next step depends on the full, validated output. Anything downstream of the model that is itself a machine.

7. Hybrid Approaches

You can stream a human-readable surface while running a parallel non-streaming call for structured extraction, or parse the stream only once a known terminator is seen. Both are more complex. Both introduce bugs. Only reach for them when the UX case is strong.

8. Token Control

Max Tokens

Caps stream length. Still the simplest lever to prevent runaway output.

Stop Sequences

Tell the decoder to halt when specific tokens appear. For JSON, a stop sequence on `"}`" can let you cut cleanly. It is brittle if your JSON contains nested objects.

Temperature

Still shapes variability. Streaming does not change what is being generated, only when you see it.

9. Failure Modes

1. Broken Output Mid-Stream

Connection drops, partial JSON, no graceful way to recover without a retry.

2. Over-Generation

Model keeps going past the useful output. Without limits or stop sequences, it can produce hundreds of unnecessary tokens.

3. Interrupt Issues

Cancelling mid-generation can leave internal state or downstream parsers in a weird place. Handle cancellation as a first-class event.

10. Subtle Insight

Streaming does not make the model smarter or faster. It only changes when the output becomes visible. The total work is identical. Use it where perception matters; skip it where correctness matters.

11. Mental Model

| **Streaming** =: *real-time token exposure, not a new capability.*

Summary

- Streaming exposes token generation as it happens.
- It improves UX but reduces control.
- It conflicts with structured outputs and tool calling.
- Best for user-facing text surfaces; avoid for machine-facing pipelines.

Chapter 10 — Batching

Batching is an efficiency idea, not an intelligence idea. It is also two different ideas sharing one name, and most of the confusion in this chapter comes from mixing them up.

1. The Core Idea

Batching means processing multiple inputs together instead of one at a time. Without batching you do request, response, request, response. With batching you send many requests, process them together, and collect many responses.

2. Why Batching Exists

LLM calls are expensive. Running them sequentially wastes time and leaves hardware underutilized. Batching parallelizes the work so compute is used efficiently.

3. Two Different Meanings of Batching

This is the main source of confusion.

1. Client-Side Batching

You, as a user of the model, send multiple independent requests in parallel.

```
Promise.all(queries.map(q => model(q)))
```

Each request is its own call. The model sees them separately.

2. Model-Side Batching

The inference system processes multiple inputs together inside one forward pass. This happens inside API providers and local inference engines. As an API user, you usually do not control this directly, but it is what makes parallel traffic fast.

4. What Happens Inside the Model

Imagine three prompts: "Hello", "Explain AI", "Write code".

Step 1: Padding

All inputs are padded to the same length so the underlying tensors line up.

Step 2: Parallel Computation

Instead of three sequential forward passes, the system runs one forward pass on a stacked input — the same matrix multiplications cover all three at once.

Step 3: Output Separation

The combined output is sliced back into individual responses.

5. Why This Is Faster

GPUs shine at large matrix multiplications. Combining many small inputs into one big tensor reduces per-request overhead and packs the hardware. Throughput goes up, often dramatically.

6. Important Clarification

Batching does not mean shared reasoning, cross-input understanding, or combined intelligence. Each input is still processed independently in terms of meaning. They just happen to share the hardware.

7. When to Use Batching

When tasks are independent, when you are repeating the same operation many times, and when there is no dependency between inputs. Typical wins: classification, embedding generation, and batch summarization of many documents.

8. When NOT to Use Batching

Avoid it when each step depends on the previous output — the classic case is an agent loop with tools and memory evolving over time. You cannot batch a ReAct cycle, because iteration two does not exist until iteration one has been observed.

9. Latency vs Throughput

Batching improves throughput (requests per second across the fleet), not latency (time for a single request). In fact, individual latency can get slightly worse inside a batch because the system may wait for enough requests to fill one. Know which metric you are actually optimizing.

10. Practical Constraints

API Models

Rate limits, token quotas, provider-side batching rules. Throttle aggressively or you will get 429s.

Local Models

Hardware limits, VRAM, batch-size sweet spots. Bigger batches help up to the point the GPU runs out of memory, then everything collapses.

11. Controlled Batching

Do not fire 1,000 requests at once. Use a batch size and a concurrency limit. Something like a queue that releases N requests at a time gives you the throughput gains without the failure modes.

12. Failure Modes

1. Over-Parallelization

Too many in-flight requests push latency up or trigger throttling. Past a point, parallelism makes everything slower.

2. Rate Limits

Providers reject excess calls. Without retry-with-backoff, you lose work.

3. Dependency Errors

Someone batches tasks that were not actually independent, and outputs end up subtly wrong in ways that are hard to trace.

13. Subtle Insight

Batching is not about intelligence. It is about work scheduling around the model. It lives in the same conceptual layer as caching and queueing — systems engineering on top of the inference call.

14. Mental Model

| **Client batching = Parallel requests:** *Model batching = Parallel computation.*

Summary

- Batching improves efficiency, not reasoning quality.
- It applies only to independent tasks.
- It trades a little latency for a lot of throughput.
- It must be controlled — unlimited parallelism fails loudly.

Chapter 11 — Error Handling

Everything in this book so far has been about making the model produce the right thing. This chapter is about what to do when it does not. In production, the second question turns out to matter at least as much as the first.

1. First Principle

Language models are unreliable components. They produce invalid outputs, make incorrect decisions, and behave inconsistently across runs. A production system has to assume failure and keep working through it.

2. Types of Failures

1. Output Failures

Invalid or malformed responses. A missing comma in JSON breaks parsing and halts the pipeline if you let it.

```
{
  "action": "search"
  "input": "laptops"
}
```

2. Decision Failures

The model chooses wrong. Wrong tool, no tool where one was needed, or a confident hallucinated answer. Syntactically valid, semantically wrong.

3. Tool Failures

External systems fail: API timeouts, network errors, malformed responses. These are not the model's fault, but they are your system's problem.

4. Loop Failures

Agent loops misbehave: infinite loops, missing stop conditions, repeated identical actions. The most dangerous category because they can run up cost quickly.

3. Validation Layer (Required)

Never trust model output. Validate structure (is this valid JSON?), required fields (is the action present?), and value correctness (is the action one of the allowed enums?).

```
if (!response.action) retry()
```

4. Retry Mechanism

If the output is invalid, ask the model to fix it. "Your previous output was invalid JSON. Return valid JSON only." Cap the retry count — two or three is normally plenty. Beyond that, you are masking a task-design problem.

5. Fallback Strategies

If retries fail, fall back. Return a safe default, simplify the task, or skip tool usage and return what you have. A fallback is not giving up; it is preferring graceful degradation over a crashed request.

6. Timeouts

External calls hang. Always define explicit timeouts — for the model call itself and for every tool call. Never allow indefinite waiting. It is how systems silently die.

7. Loop Control

In ReAct systems, cap the iteration count. If you blow past the cap, stop and return the best available result with a clear flag that it is partial. This is cheaper than letting an agent spin.

```
max_steps = 5
```

8. Guardrails

Constraints enforced at the system level, not in the prompt. Restrict which tools can be called, block unsafe arguments, enforce schemas on outputs, rate-limit on the user side. Guardrails are code, not instructions, and that is exactly why they work.

9. Common Mistakes

1. Trusting the Model

Assuming that because the output looks right, it is right. It may be right today and wrong tomorrow with the same inputs.

2. Over-Relying on Prompts

Trying to fix reliability issues by writing a cleverer system prompt. Prompts bias behavior; they do not enforce it. Enforcement has to live in code.

3. Ignoring Edge Cases

The demo works. The happy path works. The first weird input from a real user is what breaks you. Spend time thinking about the 5% of cases that will expose the system.

10. Robust System Flow

```
Input → LLM Decision → Validation → Execution → Error Handling → Output
```

Every arrow is an opportunity for something to go wrong and a place you can catch it.

11. Example Failure Handling

Step 1: the model returns invalid JSON. Step 2: validation fails. Step 3: retry with a correction prompt. Step 4: still fails. Step 5: the fallback returns a safe default, logs the failure, and the user gets a graceful message. The system continues instead of crashing.

12. Subtle Insight

Stop trying to make the model perfect. That is not a solvable problem at the model layer. Design the system to be correct on top of an imperfect generator. That is solvable, and it is your job.

13. Mental Model

| **LLM = Unreliable Generator:** *System = Reliability Layer.*

Summary

- Failures are expected, not rare.
- Validation is mandatory at every boundary.
- Retries improve reliability within limits.
- Fallbacks prevent crashes and cost explosions.
- System design must assume imperfection and absorb it.

Chapter 12 — Putting It All Together

You now have every component in this book: the model, the prompt, structured output, tools, memory, the loop, the plan, the stream, the batch, and error handling. The last move is to combine them into a working system that feels like one thing, even though it is a dozen parts.

1. From Components to System

Each chapter solved exactly one limitation of the raw model. Prompts shape behavior. JSON makes output executable. Tools give the system hands. Memory gives it continuity. Loops give it iteration. Planning gives it structure. Error handling keeps it upright. Assembled, they form an agent.

2. The Core Architecture

A minimal agent system, in plain English, is:

```
User Input → LLM Decision → Structured Output (JSON) → Validation → Execution
(tool or answer) → Update Context → Repeat or Finish
```

That is the whole spine. Everything else is a variant or a refinement.

3. The Agent Loop

```
while not done:
    response = LLM(context)
    if response.tool:
        result = execute(response)
        add result to context
    else:
        done = true
```

4. Where Each Concept Fits

LLM generates decisions. The prompt defines behavior and output shape. JSON encodes those decisions. Tools execute them. Memory maintains context across calls and sessions. The ReAct loop handles multi-step tasks. Error handling wraps every arrow to keep the system running.

5. Example Flow

User: "Find best laptops under 1 lakh and recommend one." The model decides to search. The tool executes and returns data. The model processes the results and formats a recommendation. The final answer is generated. Each step is small, structured, and testable in isolation.

6. Adding Planning

For structured tasks, wrap the loop in a plan: generate a plan, execute each step, then combine results. This is Chapter 8 plugged in as an outer layer. Predictability goes up, flexibility goes down.

7. System Variants

Simple System

Single LLM call, no tools, no memory. A chatbot. Adequate for narrow tasks with no external data.

Tool-Enabled System

LLM plus tool calling. Can look things up and take actions. Most "AI assistants" you see today live here.

Full Agent

LLM plus tools plus memory plus loop plus error handling. Handles multi-step tasks over time. Also the most engineering-heavy to build and maintain.

8. Design Tradeoffs

Flexibility vs Control

ReAct is flexible. Planning is controlled. Combine them: plan for the spine, ReAct for the joints.

UX vs Reliability

Streaming gives you better UX. Structured output gives you better control. Use streaming on human-facing surfaces and structured calls on machine-facing ones.

Performance vs Simplicity

Batching is efficient. Sequential processing is simpler and easier to debug. Start simple. Batch only where measurements say you should.

9. Common Failure Patterns

1. Over-Engineering

Adding memory, planning, and batching on day one because a tutorial said to. Most of that machinery is for later.

2. Under-Validation

Trusting model output because it usually works. It will not usually work forever.

3. Poor Tool Design

Vague names, bloated parameter lists, overlapping tools. The model cannot pick well if the options are not distinguishable.

4. Memory Misuse

Stuffing the prompt with irrelevant context, or failing to evict stale memory. More context is not more intelligence.

10. System Design Principle

Do not optimize for intelligence. Optimize for predictability, control, and reliability. A less clever system that behaves the same way every time beats a brilliant system that fails unpredictably.

11. Final Mental Model

Agent =: *LLM + Tools + Memory + Control Loop. Each component solves a specific limitation of the model.*

12. What Most People Get Wrong

They assume the model is the system. In reality, the model is one component. The system is everything around it: the prompt that shapes behavior, the schema that structures output, the tools that execute, the memory that persists, the loop that iterates, the validators that catch failures, and the fallbacks that keep things moving. Remove any of those and the system gets dumber, or fragile, or both.

13. Closing Insight

An agent is not an intelligent entity. It is a controlled feedback system built on probabilistic generation, structured constraints, and external execution. Seeing it that way strips the magic. What remains is engineering — and engineering is something you can get good at.

Summary

- Agents are composed systems, not single models.
- Each component exists because the model lacks something.
- Reliability comes from system design, not from a smarter model.
- Building agents is an engineering problem with familiar tools: validation, retries, fallbacks, logging, limits.

The cheatsheet ends here. The work begins when you wire the first JSON parser into the first model call, watch it fail, and fix it. That loop — build, observe, constrain, repeat — is the same loop the agents in this book run. Now you are running it on yourself.